# GTM Context Architecture — Plan of Record

Oct 5, 2026 · @Andy P

## Summary

The design holds up against current documentation: a local, rules-based CLAUDE.md stack that preloads interpretive GTM context, pre-declares the tool calls it needs, and routes atomic facts to governed live sources is buildable today with Claude Code, Databricks Unity Gateway, Genie, and Salesforce Hosted MCP.

The one-line thesis: **a local-first, self-governing context system that makes GTM AI predictable in cost and structurally aligned in output.**

Validation changed six things we said in conversation. Each is folded into the sections below and logged in the Validation ledger.

1. **There is no single inference call once tools run.** Claude Code sends a new model request for every step of its tool loop, re-sending the whole context each time. Prompt caching is what makes the preloaded prefix cheap on every step after the first, so cache discipline is part of the cost model.
2. **The assembled prompt does cross the gateway; the file reading does not.** When Claude Code is pointed at Unity Gateway, every model request (preloaded context included) is metered there. Reading local markdown is free.
3. **Genie is not simply a separate free budget.** Genie Agents usage by people is free until January 31, 2027, but service principals pay from the first DBU, and SQL warehouse compute is always billed separately. A scheduled batch job running as a service principal is billed.
4. **Genie MCP tool names.** The `genie_ask` / `genie_poll_response` names came from a community post. The documented Genie Agent server is read-only, carries no conversation history, and its tool names should be discovered at runtime with `tools/list`.
5. **Hook-injected context is capped.** A SessionStart hook's injected context is limited to 10,000 characters per field; overflow spills to a file Claude is not told to read. Preloaded subgraphs must be compact.
6. **Names changed.** Databricks now calls the gateway Unity Gateway (Unity AI Gateway), and Genie spaces are now Genie Agents.

## Strategic thesis

Most of the value in a sales or marketing interaction comes from context the model should already be carrying, not from querying live systems mid-task. So knowledge moves upstream into preloaded local rule files, and live lookups become the exception.

At its core this is a **rules-based CLAUDE.md architecture that runs locally**: tiered markdown files that tell the model who it serves, what matters, and how to behave, assembled on each person's machine before anything reaches the gateway. "Wiki" and "knowledge graph" describe what those files contain and how they link; the engine is a mechanism Claude Code already provides.

Three pillars carry the thesis:

- **Structural alignment.** Sales and marketing drift because they reason from different context. When every person's AI anchors on the same shared nodes, above all the campaign node, alignment is enforced by the architecture instead of by meetings.
- **Governed autonomy.** Every node carries its own rules: refresh cadence, who may change it, whether changes cascade, and what it costs to resolve. Nothing is imposed globally; people adopt updates.
- **Predictable cost.** Context is preloaded and tool calls are pre-declared, so the cost of an answer can be estimated before it runs instead of discovered afterward.

The working bet behind all three: preloaded context lets the first prompt one-shot a usable output more accurately, faster, and cheaper than retrieval-by-tool-call.

## Decision log

Twenty-seven decisions shaped the design; three were revised along the way (13, 20, 23) and one (Genie as a cost offload) needs revision after validation. Listed in the order they were made.

| # | Decision | What it replaced | Why | Status |
|---|---|---|---|---|
| 1 | Preload context locally before any gateway call | Gateway-first retrieval through many tool calls | Fewer calls, lower cost, faster first answer | Decided |

| # | Decision | What it replaced | Why | Status |
|---|---|---|---|---|
| 2 | A "soul" CLAUDE.md above all project folders; each project CLAUDE.md inherits and specializes | One flat instruction file | Shared identity and goals, per-project focus | Decided, validated |
| 3 | Must-happen work (wiki updates) lives in skills and code; judgment (when to read what) lives in CLAUDE.md | Everything as prompt text | CLAUDE.md is guidance, not enforcement | Decided, validated |
| 4 | Everyone shares a global base; each person grows a private local overlay | One shared wiki for all | Personal context without central bottleneck; privacy inherits existing access | Decided |
| 5 | Quarterly global updates applied by a person-invoked refactor skill | Silent auto-merge | People adopt updates; nothing overwritten under them | Decided |
| 6 | Cadence, mutability, and cascade are properties of each node or layer | Global refresh rules | Different knowledge moves at different speeds | Decided |
| 7 | Knowledge splits into world, market, and local layers | Undifferentiated knowledge | Each layer has its own authority and speed | Decided |
| 8 | The campaign node (Salesforce campaign = Marketo program) is the keystone | Alignment by process | Shared anchor makes sales/marketing alignment structural | Decided, keystone |

| # | Decision | What it replaced | Why | Status |
|---|---|---|---|---|
| 9 | Org layer (structure, roles) and GTM layer (accounts, campaigns, touches) are distinct, connected graphs | One graph | Different speeds and owners | Decided |
| 10 | A validated account is a persistent node refreshed daily or hourly | Static account notes | Sellers see the whole account pre-assembled | Decided |
| 11 | Two clocks: events refresh facts; the fiscal calendar re-colors interpretation | Clock-only refresh | End-of-quarter changes behavior even when nothing moved | Decided |
| 12 | Databricks is the internal CDP fed by every touchpoint, with validated crosswalk tables | Logic scattered across systems | One place to join identity, campaigns, attribution | Decided |
| 13 | Marketo is the single front door; data flows Marketo first, then native sync to Salesforce | Dual submission into both systems (corrected by Andy) | Prevents duplicates born at scattered entry points | Revised, decided |
| 14 | The crosswalk is a metadata table in Salesforce carrying IDs, lead source, next-best-action, and the attributable-event definition | Crosswalk only in Databricks | One governed row holds identity and intent | Decided |

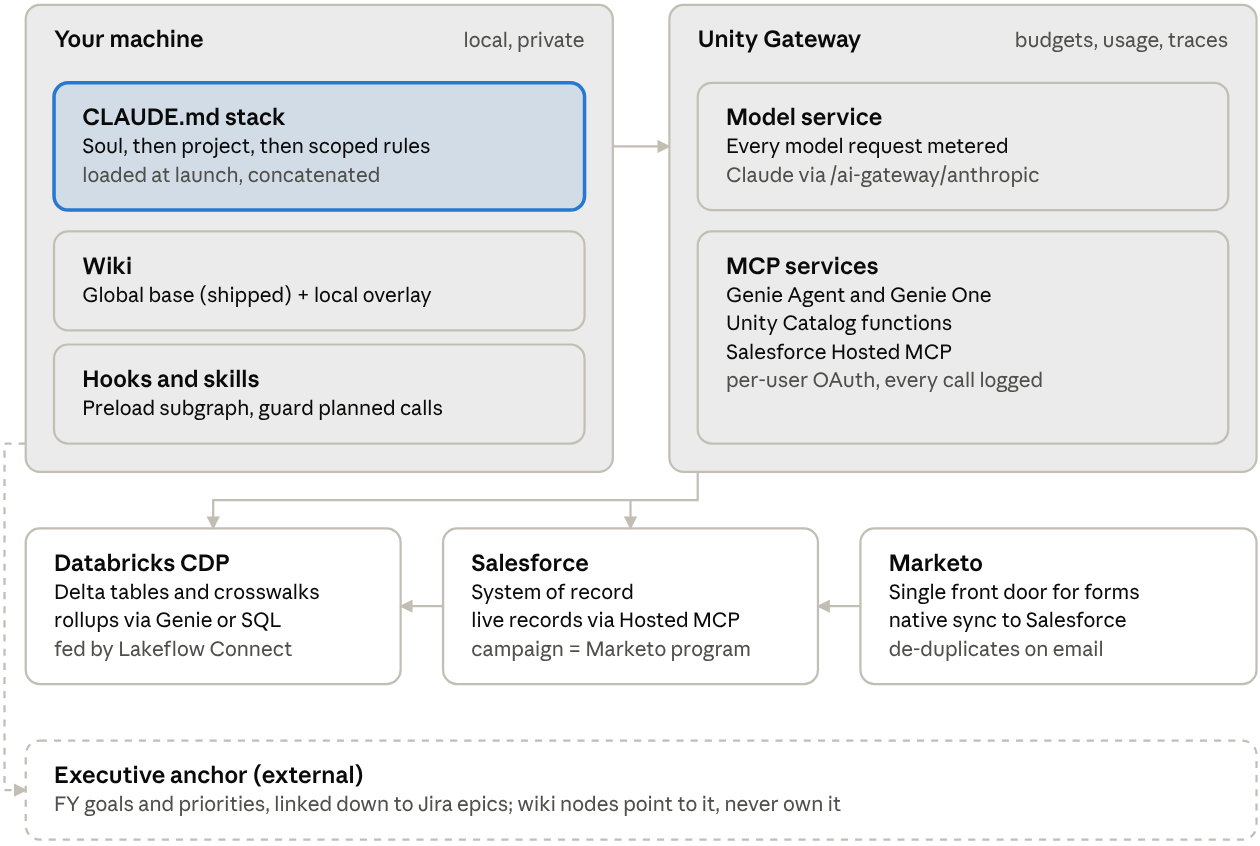
| # | Decision | What it replaced | Why | Status |
|---|---|---|---|---|
| 15 | Next-best-action fires on an attributable last touch, read from Salesforce campaign membership | On-demand queries | Customer-initiated, already the attribution system of record | Decided |
| 16 | Every campaign and signal is measured against the ICP; sales qualification tests the ICP | Volume metrics | Closes the loop on which signals matter | Decided |
| 17 | Route by data type across wiki, Genie, and gateway | Picking one retrieval method | Each method fits a different kind of question | Decided |
| 18 | Shared layers may centralize behind the gateway; personal overlay stays local | Everything local, or everything central | Shared content is identical for all; personal content is private | Proposed |
| 19 | Anything that improves first-shot quality is preloaded locally | Centralized fetch at session start | A fetch before the first token reintroduces latency | Decided |
| 20 | Interpretive context preloads; atomic facts come live | Freshness as the only axis | Business rules are the lens; facts are point-in-time | Revised into #21 |
| 21 | A grid: stable interpretive (soul), moving interpretive (market layer), atomic live (gateway) | Binary interpretive vs atomic | Fast-moving targets still need preloading at their own cadence | Decided |

| # | Decision | What it replaced | Why | Status |
|---|---|---|---|---|
| 22 | Preloaded markdown carries a plan of pre-declared tool calls, each mapped to a skill | Model improvises tool calls | Bounded calls make cost estimable per task | Decided |
| 23 | Executive goals become an external anchor the wiki points to, linked down to Jira epics | Executive layer inside the stack | It never behaved like GTM knowledge; it is org state over time | Revised, decided |
| 24 | The wiki stores query intent and interpretation, not live Salesforce rows | Copying CRM data into the wiki | Live data goes stale instantly; no second system of record | Decided |
| 25 | Aggregates go to Databricks; live single-record lookups go to Salesforce | One path for all Salesforce data | Freshness matters for one record, rarely for a rollup | Decided |
| 26 | Genie runs as a scheduled batch engine on a separate budget | Genie called live during the one-shot | Async polling suits batch; moves spend off the gateway | Needs revision (billing) |
| 27 | Live and local paths are self-gating by OAuth; the shared layer must be permission-free by construction | Trusting the shared layer | A node built from restricted data would leak it | Decided |

## Architecture overview

Everything that shapes reasoning is assembled on the person's machine; only model requests and planned tool calls leave it, and both pass through one governed gateway.

**Context is assembled locally; only governed calls leave the machine**




System architecture · local assembly, one governed gateway, three sources

Read it left to right, then down. The CLAUDE.md stack, wiki, and hooks build the prompt locally; Unity Gateway meters each model request and brokers every tool call under the caller's identity; data comes from the Databricks CDP for rollups and from Salesforce for live records, with Marketo feeding Salesforce through the native sync. The dashed box is the executive anchor, which wiki nodes point to without owning.

## The CLAUDE.md tier stack

The tier design maps directly onto how Claude Code loads instructions: every CLAUDE.md from the filesystem root down to the working folder loads at launch and is concatenated, broader first, so a soul file placed above all project folders is inherited by every project without any import.

| Tier | File and location | When it loads | What it holds |
|---|---|---|---|
| Company policy (optional) | Managed CLAUDE.md deployed by IT (macOS: `/Library/Application Support/ClaudeCode/CLAUDE.md`) | Every session on the machine; cannot be excluded | Compliance and data-handling rules every GTM user must follow |
| 1 Soul | `~/gtm/CLAUDE.md`, the folder above all project spaces (or `~/.claude/CLAUDE.md` for every project on the machine) | At launch, for any project under it | Mission, goals, values, primary sources, people in daily orbit, routing order (preload → local → gateway) |
| 2 Project | `~/gtm/<project>/CLAUDE.md` | At launch when working in that project | Project goals, conventions, state, which wiki slices and pre-declared calls apply |
| 2b Personal per project | `CLAUDE.local.md` beside it, gitignored | At launch, after the project file | Private notes for that project |
| 3–4 Scoped knowledge | `.claude/rules/*.md` with `paths:` frontmatter, or a subfolder's own CLAUDE.md | Only when Claude reads or edits a matching file | Account, campaign, or segment slices of the graph |

Five documented behaviors shape the build:

- **Size.** Target under 200 lines per CLAUDE.md; longer files cost context and lower adherence. `@path` imports organize a file but do not reduce cost, because imports also load at launch (up to four hops deep).
- **Scoped loading is native.** Path-scoped rules and subfolder CLAUDE.md files load on demand. That is the built-in mechanism for relevance-scoped preloading: put each account's node under `accounts/<account>/` and its rules load only when that account is touched.

- **Guidance, not enforcement.** CLAUDE.md arrives as a user message after the system prompt. Anything that must happen belongs in a hook: a SessionStart hook can inject a computed subgraph as `additionalContext` (capped at 10,000 characters per field) and a PreToolUse hook can deny a tool call outside the plan.
- **AGENTS.md.** By default Claude reads AGENTS.md only when no CLAUDE.md exists on the path. For colleagues on other agents, keep one AGENTS.md and import it from CLAUDE.md with `@AGENTS.md`.
- **Plugins cannot ship CLAUDE.md files.** A plugin bundles skills, agents, hooks, and MCP servers. That confirms the bootstrap skill's job: it must write the soul and project files itself, and merge into existing ones without clobbering them.

```
~/gtm/
├── CLAUDE.md                    # Tier 1 soul (loads for every project
below)
├── wiki/                        # Tier 3 global base (shipped) + Tier 4
local overlay
|   ├── market/campaigns/        # campaign nodes (keystone)
|   ├── market/accounts/<acct>/  # persistent account nodes
|   └── local/                   # private; promoted only through review
├── account-briefs/
|   ├── CLAUDE.md                # Tier 2 project
|   └── .claude/rules/acct.md    # paths: frontmatter, loads only on
matching files
└── campaign-planning/
    └── CLAUDE.md
```

One layout choice is still open. Path-scoped rules and subfolder CLAUDE.md files trigger on files inside the working folder. A wiki that sits beside the projects is reachable with `--add-dir`, but its CLAUDE.md files load only when `CLAUDE_CODE_ADDITIONAL_DIRECTORIES_CLAUDE_MD=1` is set. The bootstrap skill should either set that variable or place the wiki inside each project folder.

## Knowledge layers and governance

Position in the stack is the governance: slow and authoritative layers sit high and load whole, fast and personal layers sit low and load as scoped slices. Each node carries its own cadence, mutability, cascade rule, and cost to resolve.

| Layer | Contents | Refresh | Who may change it | Cascades? | How it preloads |
|---|---|---|---|---|---|
| World | Forces outside the company: regulation, macro, category shifts | Slow, on review | Curators of the global base | No; adopted at quarterly refactor | Small, loads whole |
| Org | Structure, roles, teams, org chart, Jira mapped to owners | Quarterly, or on reorg | Global base owners | Adopted at refactor | Loads whole if small, else by team |
| Market: campaigns | Campaign nodes from the crosswalk: IDs, intent, attributable event, next-best-action | Daily, machine-validated against Salesforce | System of record only | Global updates daily; users adopt | The campaigns a task touches |
| Market: accounts | Persistent account nodes: who is involved, touches, open opportunities, stage | Event-driven, at least daily, possibly hourly | System of record plus validated enrichment | Updates the node, not users' notes | One account subgraph per task |
| Local overlay | A person's own Salesforce, Jira, Slack, Confluence, Drive context | Continuous, on the person's machine | That person | Never leaves the machine unless promoted | Scoped by project |
| Calendar clock | Fiscal period, quarter-end, cycle stage | Calendar events | Finance / leadership | Re-colors interpretation company-wide | Always, a few lines |

Three rules keep the layers honest:

- **Two clocks.** Events (an attributable touch, a stage change) refresh facts in the GTM layer. The fiscal calendar changes how those facts should be read, such as quarter-end urgency, even when nothing on the account moved.
- **Validated vs conjecture.** Nodes and edges carry a validation flag. Strong, sourced connections are facts; predictive connections are shown as worth investigating, never as truth.
- **Refactor records lineage.** Each local node records which global node and version it attaches to, so the quarterly refactor can reconcile instead of overwrite.

## Executive layer: an external anchor

Decided: executive direction is not a layer inside the GTM wiki; it is an external anchor the wiki points to. It was the one "layer" that never behaved like the others: immutable, set by leadership on its own clock, never refactored by users.

It holds two different things, and they connect:

- **Strategic intent:** fiscal-year goals, mission, priorities, roadmap items, what the company hopes to build.
- **Tangible execution:** the work employees produce against that intent, tracked as Jira epics and tickets.

The relationship is a linking cascade, not a rewriting one. A campaign, account play, or epic node carries a pointer such as `serves: FY27-priority-2`, so any output traces up to the goal it moves. The goal itself is never copied into or governed by the wiki.

Why it matters for GTM: this context is mostly "what is happening in the company over time." It informs reasoning when relevant but does not need to be preloaded into every GTM prompt. Load the anchor's one-line summary in the soul; fetch detail only when a task names a priority.

## Retrieval routing

Route by the nature of the knowledge, not by picking one method: the wiki holds the why and what-matters, Databricks holds rollups, and Salesforce holds the precise current fact.

| Kind of knowledge | Example | Lives in | Reaches the model by | Fresh as |
|---|---|---|---|---|
| Stable interpretive | Mission, values, ICP definition, what "good" looks like | Soul and global base | Preloaded at launch | Quarterly refactor |
| Moving interpretive | This quarter's pipeline target, a campaign's conversion threshold, this month's definition of qualified | Market layer | Preloaded at launch, refreshed on its own cadence | Daily |

| Kind of knowledge | Example | Lives in | Reaches the model by | Fresh as |
|---|---|---|---|---|
| Aggregates | Pipeline rollups, campaign performance, trend over time | Databricks CDP tables | A pre-computed snapshot node preloaded; live Genie only when a task needs a new cut | Last pipeline run |
| Single live record | Exact stage or amount on one opportunity right now | Salesforce | Live tool call via Salesforce Hosted MCP, last resort | Real time |
| Personal | My Slack thread, my notes on an account | Local overlay | Scoped preload by project | Continuous |

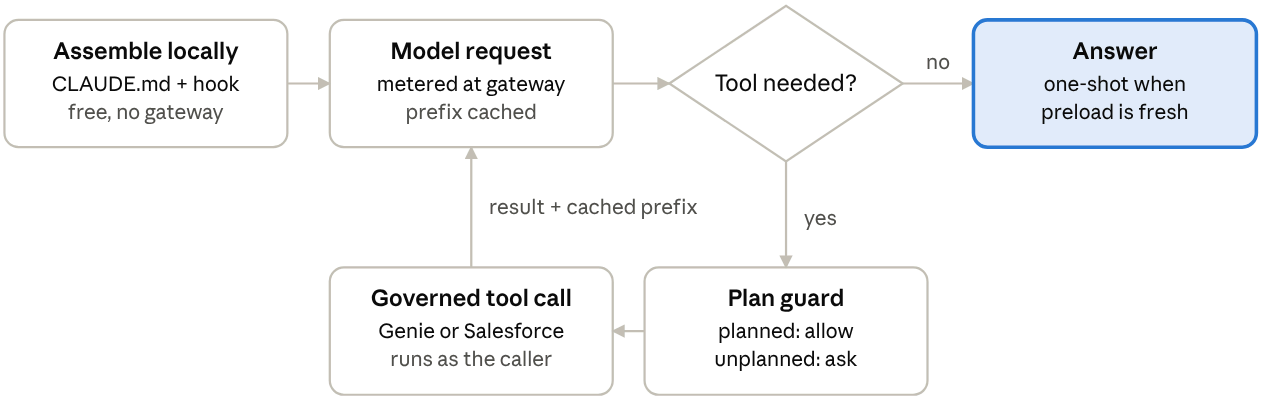
The wiki never stores live Salesforce rows. It stores **query intent and interpretation**: which object and fields to fetch for a task, which path to send it down, and how to read what comes back.

Lookup order, written into the soul file: preloaded context → local wiki lookup → Databricks (Genie Agent or a Unity Catalog function) → live Salesforce through the gateway.

## Request lifecycle through the gateway

Yes, the prompt goes through the gateway, but reading the markdown does not: Claude Code assembles the context on the machine, then every model request (preload included) is metered at Unity Gateway, once per step of the tool loop.

**Every tool round re-sends the context; caching makes repeats cheap**




Request lifecycle · local assembly, metered requests, guarded tool loop

When the preload is fresh, the loop ends at the first request and the answer is a one-shot. Each tool round sends another request carrying the same prefix, which is billed at the cache-read rate if it has not changed and the cache has not expired.

How Claude Code is pointed at the gateway, per the Databricks documentation: run `ug claude`, or set `ANTHROPIC_BASE_URL` to `https://<workspace>/ai-gateway/anthropic` in `~/.claude/settings.json` together with a Databricks token, the model's Unity Catalog name, and `ENABLE_PROMPT_CACHING_1H=1`. MCP services are added with `ug mcp add` or `claude mcp add --transport http` against `/ai-gateway/mcp-services/<catalog>.<schema>.<service>`.

## Wiki as planner and the cost model

The preloaded markdown carries a plan of investigation as well as knowledge: for each task, the exact tool calls that may be needed, each mapped to a skill, each with a condition and a budget. A bounded plan turns cost from a random variable into a sum you can estimate before running.

A plan block inside a project's rules might read:

```
task: account-briefing
preload: [soul, campaigns/<active>, accounts/<acct>, calendar]
calls:
  - id: opp-status
    skill: sf-live-record          # Salesforce Hosted MCP, sobject-reads
    when: account node older than 24h OR opportunity missing
    budget: 1 call
  - id: touch-rollup
    skill: genie-campaign-touches  # Genie Agent over CDP tables
    when: no touch snapshot in last 7 days
    budget: 1 call
escape_hatch: ask before any unplanned call
```

The cost of one task, given that Claude Code makes one model request per step of its tool loop:

```
\text{Cost} \approx w \cdot P + (S-1)\, r \cdot P + \sum_{s=1}^{S}\left( i
\cdot N_s + o \cdot O_s \right) + \sum_{c=1}^{K} \text{tool}_c
```

where P is the preloaded prefix in tokens, S the number of model steps (one plus the number of tool-call rounds), N and O the new input and output tokens per step, K the planned calls, and w, r, i, o the per-token prices for cache write, cache read, input, and output. On the Claude API a 5-minute cache write costs 1.25× base input, a 1-hour write 2×, and a cache read 0.1× (0.05× on Opus 5.5). Tool costs are Genie DBUs plus SQL warehouse time, or Salesforce API requests.

What this means in practice:

- **Every tool round re-sends the preload.** That is why caching matters more than prefix size: after the first step the preload is billed at the read rate, so a large stable prefix is cheap and a prefix that changes mid-session is expensive.
- **Keep the prefix byte-stable within a session.** Refresh nodes between sessions, not during them; any change to earlier content invalidates the cache from that point on.
- **Resumed sessions pay again.** A cache expires after 5 minutes by default. Databricks' documented Claude Code settings for Unity Gateway turn on the 1-hour cache (ENABLE_PROMPT_CACHING_1H=1), which costs 2× base input to write but survives longer pauses. Claude Code's SessionStart hook reports `prompt_cache_likely_expired` and `estimated_cache_write_usd` on resume, which a hook can surface before the first request.

- **Conditional calls keep most tasks at zero calls.** The `when:` clause means a fresh preload satisfies the task with S = 1.
- **Estimate before running.** A hook can sum the planned calls' budgets and show the expected cost; a skill can ask for confirmation above a threshold.
- **Enforce the plan, keep an escape hatch.** A PreToolUse hook can match each MCP tool name against the plan and return `ask` for an unplanned call, so the model flags a gap instead of silently proceeding or silently spending.
- **Measure after.** Model tokens per request land in `system.ai_gateway.usage`; Genie usage lands in `system.billing.usage` with `billing_origin_product = 'GENIE'`. Since September 2026 the gateway usage table also records coding-agent session metadata and tokens written to the 5-minute and 1-hour caches, and unified tracing shows the tools and skills used alongside each model request. Joining these to the plan's task id gives cost per task and per call.

## Genie: invocation, billing, role

Genie can be invoked by Claude as a tool today, but the cost offload only works when a person is the caller: Genie Agents usage by users is free until January 31, 2027, while service principals pay from the first DBU and SQL warehouse compute is always billed.

### Three ways to invoke it

| Path | Endpoint | Behavior | Best for |
|---|---|---|---|
| Genie Agent managed MCP server | `https://<workspace>/api/2.0/mcp/genie/{genie_space_id}`, OAuth scope `genie` | Read-only; one question in, SQL run, result back; no conversation history passed | Claude Code calling one curated agent of up to 50 tables |
| Genie One (MCP Service in Unity Gateway) | `https://<workspace>/ai-gateway/mcp-services/system.ai.genie_one_mcp`, scope `ai-gateway` | Searches across Genie Agents and Unity Catalog, returns a grounded answer | Broad questions when the right agent is unknown |
| Genie Conversation API | `POST /api/2.0/genie/spaces/{space_id}/start-conversation`, then poll the message, then fetch the result | Asynchronous; stateful follow-ups; poll with exponential backoff up to one minute | Scheduled jobs and custom code |

Tool names on the MCP servers should be read at runtime with `tools/list` rather than hard-coded. Databricks recommends Genie One over the Databricks SQL MCP server for analytics, because Genie resolves business terms through its governed semantic layer.

**How it is billed**

| Cost | What it covers | Free? |
|---|---|---|
| Genie LLM usage | The model that turns a question into SQL | 150 DBUs per user per month (about $10.50 in US East); Genie One and Genie Agents free for users through January 31, 2027; service principals billed from the first DBU |
| SQL warehouse compute | Running the generated query | Never free; billed as Databricks SQL |
| Unity Catalog functions | Predefined SQL run as a tool | Serverless general compute |

Genie spend is tracked under `billing_origin_product = 'GENIE'` in `system.billing.usage`, with the product surface in `usage_metadata.genie.surface` and the caller in `identity_metadata.run_as`. Budgets are set on the Unity Gateway

resource type with the `databricks-product: genie` tag and track LLM usage only, not warehouse compute.

**Revised role**

- **Scheduled pre-compute stays the right shape**, because async polling suits batch and the output lands in the preload. But a job running as a service principal is billed for Genie LLM usage. For truly fixed queries, skip Genie and run plain SQL or a Unity Catalog function on a schedule: no LLM cost, deterministic output.
- **Live Genie calls from Claude Code run as the person**, so through January 2027 they draw on free user usage; only warehouse compute is paid. That makes Genie a reasonable on-demand path for new cuts of data.
- **Deterministic tools beat open questions for predictable cost.** A Unity Catalog function with fixed SQL has a known cost per call; a Genie question does not. Pre-declared plan calls should prefer functions where the query is known in advance.

## Salesforce: live path vs Databricks copy

Send aggregates to the Databricks copy and single current-record lookups to Salesforce live; the live path now has a governed, per-user option in Salesforce Hosted MCP servers.

| | Live: Salesforce Hosted MCP | Copy: Databricks CDP |
|---|---|---|
| How | `sobject-reads` server at `https://api.salesforce.com/platform/mcp/v1/platform/sobject-reads`: discovery, query, search, relationship traversal, no writes | Lakeflow Connect Salesforce connector lands objects as Delta tables; queried by Genie, SQL, or Unity Catalog functions |
| Freshness | Real time | As fresh as the pipeline schedule; formula fields need a full snapshot by default, which adds latency |
| Permissions | Per-user OAuth 2.0 with PKCE; respects field-level security and sharing rules | Unity Catalog grants on the tables; must be designed to match Salesforce visibility |

| | Live: Salesforce Hosted MCP | Copy: Databricks CDP |
|---|---|---|
| Cost and limits | Plan as if it counts toward the org's daily API allocation (to confirm): 100,000 plus 1,000 per Salesforce license per rolling 24 hours on Enterprise Edition | Warehouse compute; no Salesforce API load at query time |
| Setup | Enable MCP Service, create an External Client App with the MCP scope; Enterprise Edition and above | Connector OAuth uses a manual refresh token; rotated tokens must be re-entered |
| Use for | "What is the stage on this opportunity right now?" | Pipeline rollups, campaign performance, trends |

Two details from the connector documentation to design around: hard deletes on soft-delete tables are not captured, and non-formula fields can update earlier than formula fields within the same cadence. Rollups that depend on formula fields should note their as-of time.

The Salesforce query language is SOQL. With the Hosted MCP server, Claude generates and runs queries under the user's identity; the plan file should still name the object, fields, and filter for each pre-declared call so the query shape, and its cost, are fixed.

Whether Salesforce API usage is counted per call through MCP the same way as REST calls is not stated on the overview page; confirm with your Salesforce admin before relying on live volume.

## Identity and permissions

The permission model is mostly inherited, not built: live calls run as the caller and local data never leaves the machine. The one place it must be designed is the shared global layer.

| Path | Enforcement | Documented basis |
|---|---|---|
| Local overlay | Physical: the data sits on the person's machine | Claude Code reads local files; nothing is sent until a model request |

| Path | Enforcement | Documented basis |
|---|---|---|
| Databricks managed MCP (Genie, UC functions, SQL) | On-behalf-of-user OAuth; Unity Catalog permissions enforced on every request | Managed MCP servers automatically respect a user's permissions |
| Unity Gateway MCP Services (Slack, Drive, registered external servers) | Caller must hold `EXECUTE` on the service plus `USE CATALOG` and `USE SCHEMA`; per-user OAuth U2M available; every call logged | MCP Services docs |
| Salesforce Hosted MCP | Per-user OAuth with PKCE; field-level security and sharing rules apply | Salesforce Hosted MCP docs |
| Shared global layer | Nothing automatic: the same nodes ship to everyone | Must be designed |

**The shared-layer rule.** A node may enter the global base only if its content is safe for every person who will receive it. A summary built from a restricted opportunity would carry that data to people who cannot query the source. Anything permission-sensitive stays in the local overlay or behind a live, gated call, where OAuth checks again every time.

So promotion from local to global is two tests, not one: is it validated, and is it permission-free for all recipients. The promotion skill should check both.

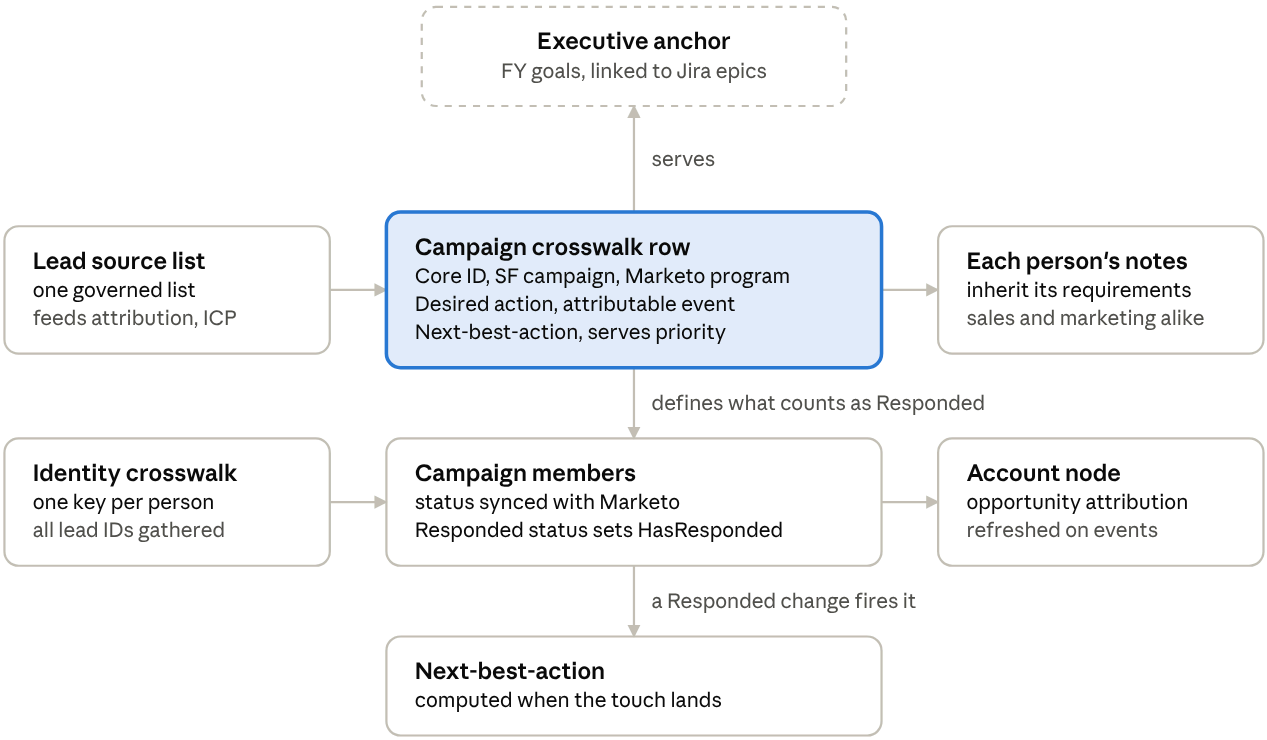
Operational notes from the docs:

- **Coding agents connect with OAuth or a PAT.** Databricks recommends OAuth for team use; the Unity Gateway CLI (`ug`) authenticates through the Databricks CLI login and configures Claude Code's models, MCP servers, and skills in one step.
- **No dynamic client registration.** Databricks MCP endpoints do not support it, so an admin creates the OAuth app (client ID, redirect URL, scopes) once.
- **Workspace IP access lists** must allow the client's outbound addresses if they are enabled.
- **Per-user consent is one time.** Each user completes an OAuth login the first time they call a per-user MCP Service.

# Data model: the campaign keystone

One governed campaign row holds both identity and intent: the IDs that tie Salesforce to Marketo, the action the campaign wants the target to take, the event that proves they took it, and what to do next. Everyone's local knowledge built on that row inherits the same strategic requirements.

**One campaign row anchors identity, intent, and the next action**




Campaign keystone · one row joins identity, intent, trigger, and action

The accented row is the keystone. Lead source and identity feed it from the left; each person's notes inherit from it on the right; its definition of Responded turns a campaign member's status change into the trigger for next-best-action.

### Campaign crosswalk, one row per campaign

| Field | Example | Notes |
|---|---|---|
| Core campaign ID | `FY27-WEB-014` | Company-specific key |
| Salesforce Campaign ID | `701…` | Join to CampaignMember and opportunity attribution |

| Field | Example | Notes |
|---|---|---|
| Marketo Program ID | `4521` | Marketo syncs one program to exactly one campaign |
| Lead source | `Web Form` | From the governed lead-source list |
| Desired action | Download the guide through the gated form | What the campaign asks the target to do |
| Attributable event | Member status Sent → Responded, triggered by a form fill | Fires next-best-action |
| Next-best-action | Route to SDR within 24 hours with talk track B | Read directly by the engine |
| Serves priority | `FY27-priority-2` | Pointer to the executive anchor |

**Two supporting tables**

- **Identity crosswalk.** Many duplicates share an email, and email alone is not enough. Build one universal person key that gathers every lead ID and contact ID ever attached to a person, plus other identifiers.
- **Lead source dimension.** Every lead entering Salesforce has a lead source; it cuts across leads and campaigns. Keep one centrally governed canonical list to avoid spelling drift. It feeds attribution and the ICP loop.

**Where the crosswalk lives.** Decided: a metadata table in Salesforce, mirrored to Databricks. Salesforce offers two shapes. Custom Metadata Types are deployable configuration, and queries against them in Apex do not count toward SOQL governor limits, but records change by deployment. A custom object changes by ordinary record edits, which suits campaigns created weekly. Pick by how often marketing ops creates campaigns.

**What the systems guarantee (validated)**

- Marketo maps one program to exactly one Salesforce campaign; program statuses must match campaign member statuses before sync, and member status changes sync both ways.
- Marketo channel and period cost sync one way to Salesforce Campaign Type and Actual Cost.
- If a program member does not exist in Salesforce, Marketo creates the lead there.

- Salesforce marks a campaign member as responded (`HasResponded = true`, plus `FirstRespondedDate`) when its status is one of the values flagged as Responded for that campaign. That flag is the clean, native definition of an attributable event.
- Program-to-campaign sync runs through a queue on a cycle of roughly seven minutes per community reports, so "real time" next-best-action means within minutes, not seconds.

**Ingestion: Marketo is the single front door.** All forms are Marketo forms; data enters Marketo first, then flows to Salesforce through the native sync (not dual submission). Marketo de-duplicates people by email on form fills. It can still create duplicates through API inserts, CRM inserts, or forms without an email field across browser sessions, so the identity crosswalk remains necessary.

## Next-best-action engine

Next-best-action is attached to every attributable touchpoint, fired by a customer's own action, and reasoned over the account's whole history: last-touch triggered, full-history informed.

1. **Trigger.** A campaign member's status changes to a value flagged as Responded on that campaign (Salesforce sets `HasResponded`). Because every attributable last touch at the company runs through a Salesforce campaign, campaign membership is the trigger vocabulary.
2. **Look up intent.** The engine reads the campaign's crosswalk row: the desired action, the attributable-event definition, and the configured next-best-action.
3. **Assemble context.** The person's account node, its strong validated connections, the campaign node, the calendar clock, and the executive anchor pointer are loaded as one scoped subgraph.
4. **Recommend.** The model applies the configured action to this account's arc: what has happened, who is involved, where the opportunity stands, what quarter it is.
5. **Measure.** The touch and its outcome are scored against the ICP. Sales qualification feeds back whether the ICP and the high-intent signals (including Marketo signals) are right.

The trigger lives in the market layer and refreshes on events. The recommendation is computed when the touch arrives, not on request, so a seller opens the account and the next step is already there.

# Plugin and skills build list

Ship one Claude Code plugin that bundles the skills, two hooks, and the MCP connections; the bootstrap skill writes the CLAUDE.md tiers because plugins cannot carry them.

| Component | Job | Mechanism | Origin |
|---|---|---|---|
| Bootstrap | Stand up the whole environment from zero, idempotently: project space, routines, soul and project CLAUDE.md; merge into existing files without clobbering; light onboarding to fill the soul | Skill that writes files | Andy |
| Source connectors | Plug a person's authorized Salesforce, Jira, Slack, Confluence, Drive into the local overlay | Skills plus MCP servers | Andy |
| Wiki update | Capture raw, then a curation pass decides what graduates into the wiki | Skill (must-happen work in code) | Andy |
| Quarterly refactor | Reconcile the local graph against the updated global base, person-invoked | Skill using each node's lineage | Andy |
| Account briefing | Everything known about an account, against the whole wiki, usable on a call or for a campaign | Skill with a pre-declared call plan | Andy |
| Validated vs conjecture | Flag unvalidated predictive connections as worth investigating | Skill | Andy |

| Component | Job | Mechanism | Origin |
|---|---|---|---|
| Next-best-action | Compute the recommendation when an attributable touch arrives | Skill reading the crosswalk row | Andy |
| Preload hook | Inject the scoped subgraph for the task at session start | SessionStart hook, `additionalContext` under 10,000 characters | Proposed |
| Plan guard | Allow planned calls, ask on unplanned ones, log cost per call | PreToolUse hook matching MCP tool names | Proposed |
| Cost estimator | Sum the plan's budgets and show expected cost before running | Skill or hook | Proposed |
| Promotion | Nominate a local node to the global core; checks validation and permission-free content | Skill | Andy (later), test added |
| Provenance | Tag each node with its source; flag unsourced or AI-generated nodes | Skill | Proposed |
| Dedup and merge | Consolidate near-duplicate nodes | Skill | Proposed |
| Freshness | Flag nodes stale against their upstream source | Skill | Proposed |

### Two distribution paths, both documented

- **Claude Code plugin marketplace.** A repository with `.claude-plugin/marketplace.json`; colleagues install with one command and get updates. Managed settings can allowlist the marketplace or force-install the plugin.
- **Unity Gateway coding-agent configuration.** Admins publish models, MCP servers, and skills under Govern → Coding Agents, and `ug` applies the configuration on each developer's machine. This requires the Enhanced Unity Gateway preview, and Unity Gateway skills are in Beta.

The gateway path centralizes governance and spend tracking; the plugin path also carries hooks, while the gateway configuration is documented for models, MCP servers, and skills only. A practical split: the plugin for skills and hooks, the gateway for model routing and MCP services.

## Validation ledger

Of 20 technical claims made in conversation, 12 are confirmed, 5 are confirmed with a correction, 2 need a correction, and 1 is unverified.

| # | Claim from the conversation | Verdict | What the documentation says | Source |
|---|---|---|---|---|
| 1 | Claude Code fills the context from local markdown before anything is sent | Confirmed | CLAUDE.md files from the root down to the working folder load at session start; reading files is local | Claude Code memory |
| 2 | A soul file above project folders is inherited by every project | Confirmed | Ancestor CLAUDE.md files load at launch and concatenate, broader first | Claude Code memory |
| 3 | CLAUDE.md is the auto-load filename; AGENTS.md is a different convention | Confirmed, with update | Claude Code now reads AGENTS.md when no CLAUDE.md exists, or both if configured | Claude Code memory |
| 4 | Must-happen work belongs in code, judgment in CLAUDE.md | Confirmed | CLAUDE.md is context, not enforcement; use hooks for what must happen | Claude Code memory, hooks |
| 5 | Relevance-scoped preloading of a subgraph | Confirmed | Path-scoped rules and subfolder CLAUDE.md load on demand; SessionStart `additionalContext` is capped at 10,000 characters per field | Claude Code memory, hooks |
| 6 | The prompt passes through the AI gateway | Confirmed | Claude Code pointed at `/ai-gateway/anthropic` routes every model request through Unity Gateway, with token usage per user | Connect Claude Code, usage tracking |

| # | Claim from the conversation | Verdict | What the documentation says | Source |
|---|---|---|---|---|
| 7 | Preloading is free in gateway terms; one fat inference call | Corrected | Reading is free, but every step of the tool loop re-sends the context; prompt caching makes repeats cost 0.1× (0.05× on Opus 5.5) | Prompt caching |
| 8 | Genie can be invoked as a tool | Confirmed | Genie Agent managed MCP server, Genie One MCP Service, and the Conversation API | Managed MCP servers |
| 9 | Genie Conversation API is async: ask, poll, fetch | Confirmed | Start a conversation, poll every 1 to 5 seconds with backoff up to one minute, stop after 10 minutes, fetch the result; 200,000 conversations per agent | Genie Agents API |
| 10 | Managed MCP tools are named `genie_ask` and `genie_poll_response` | Unverified | Came from a community post; official docs say to discover tool names with `tools/list` | MCP Services |
| 11 | Genie draws on a separate budget, so it moves spend off the gateway | Confirmed, with correction | Separate meter (`billing_origin_product = 'GENIE'`); free for users through January 31, 2027; service principals billed; SQL warehouse always billed | Monitor Genie cost, Genie budgets |
| 12 | Gateway tools inherit each user's access via OAuth | Confirmed, with correction | True for managed MCP (on-behalf-of-user) and per-user MCP Services; MCP Services can also be configured with a shared principal, which would not inherit access | MCP Services |
| 13 | Live Salesforce data can be pulled with SOQL | Confirmed | Salesforce Hosted MCP servers (reported generally available since April 2026; confirm in your org) use per-user OAuth; respect field-level security and sharing | Hosted MCP Servers |
| 14 | Staging in Databricks introduces freshness lag | Confirmed | Lakeflow Connect formula fields need a full snapshot by default, adding latency | Salesforce connector limits |

| # | Claim from the conversation | Verdict | What the documentation says | Source |
|---|---|---|---|---|
| 15 | Live Salesforce queries carry load and limits | Confirmed | Enterprise Edition: 100,000 plus 1,000 per license API requests per rolling 24 hours | Salesforce API limits |
| 16 | A Marketo program and Salesforce campaign are one object pair | Confirmed | One-to-one mapping; statuses must match; member statuses sync both ways | SFDC campaign sync |
| 17 | The attributable event is a status change such as Sent → Responded | Confirmed | `HasResponded` is set by statuses flagged as Responded; `FirstRespondedDate` records when | CampaignMember |
| 18 | Marketo as single front door prevents duplicates | Confirmed, with correction | Form fills de-duplicate on email; API and CRM inserts can still create duplicates | Marketo community |
| 19 | The crosswalk can live as metadata in Salesforce | Confirmed, with tradeoff | Custom Metadata Types deploy as config and skip Apex SOQL limits; custom objects suit frequent edits | Custom Metadata Types |
| 20 | Genie spaces and the AI Gateway | Corrected (names) | Now Genie Agents and Unity Gateway; Genie One and Genie Code MCP connectors moved to Unity Gateway | Unity Gateway release notes |

## Open questions and next steps

The first build step is confirming how your own Unity Gateway is configured, because three cost and permission assumptions depend on it.

### Questions for the Databricks and Salesforce owners

- [ ] Is Claude Code routed through Unity Gateway with `ug` or manual settings, and is the 1-hour prompt cache enabled?
- [ ] Which model services and budgets apply to GTM users, and who can read `system.ai_gateway.usage`? (Account admins only by default.)
- [ ] Will Genie calls from Claude Code run as each user (free through January 31, 2027) or through a service principal (billed)?

- [ ] Are the MCP Services we need configured per-user (inherits access) or with a shared principal (does not)?
- [ ] Is Salesforce Hosted MCP enabled, and do its calls count against the daily API allocation?
- [ ] Is a Lakeflow Connect pipeline landing Campaign, CampaignMember, Lead, Contact, Account, and Opportunity, and on what schedule?
- [ ] Does the crosswalk live as Custom Metadata or a custom object, given how often campaigns are created?

**Build order**

1. Write the soul CLAUDE.md and one project CLAUDE.md by hand; measure them against the 200-line target.
2. Stand up the campaign crosswalk with five real campaigns, including the Responded status definition for each.
3. Build the account-briefing skill with a two-call plan (one Salesforce live record, one Databricks rollup) and the PreToolUse plan guard.
4. Run ten briefings; read cost per task from the gateway usage table and Genie billing rows; compare against the estimate.
5. Generalize into the bootstrap skill and the plugin; decide plugin marketplace vs Unity Gateway distribution.
6. Add the next-best-action trigger on Responded status, then the quarterly refactor and promotion skills.

## Sources

Anthropic

- How Claude remembers your project (CLAUDE.md, rules, AGENTS.md)
- Hooks reference
- Plugins overview
- Prompt caching

Databricks

- Databricks managed MCP servers
- Genie Agent MCP server
- Use the Genie Agents API

- Connect MCPs to AI assistants and coding agents
- Connect agents to tools with MCP Services
- Connect Claude Code to Unity Gateway
- Unity Gateway release notes
- Monitor AI Gateway usage
- Monitor and understand your Genie cost
- Manage budgets for Genie
- Genie pricing
- Salesforce ingestion connector limitations

Salesforce and Marketo

- Salesforce Hosted MCP Servers
- Salesforce Hosted MCP server endpoints (LibreChat setup guide)
- API request limits and allocations
- CampaignMember object reference
- Custom Metadata Types guide (Apex Hours)
- Marketo SFDC sync: campaign sync
- Marketo community: implications of duplicate records
- Marketo community: program and campaign sync timing